AI CHATBOT FOR COLLEGE ENQUIRY

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA1711 - Artificial Intelligence

to the award of the degree of

BACHELOR OF ENGINEERING

IN

Cyber Security

Submitted by

Thanigaivel S (192565006)

Ranjith M (192511179)

Under the Supervision of

Dr. Krishnakumar

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "AI Chatbot for College Enquiry" has been carried out by Thanigaivel S (Reg. No. 192565006) and Ranjith M (Reg. No. 192511179) under the supervision of Dr. Krishnakumar and is submitted in partial fulfilment of the requirements for the current semester of the B.E. / B.Tech. Computer Science and Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR	**COURSE FACULTY**
Title / Name,	Title / Name,
Designation	Designation
Department of CSE	Department of CSE
SIMATS Engineering, Chennai - 602105.	SIMATS Engineering, Chennai - 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, Thanigaivel S (192565006) and Ranjith M (192511179) of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "AI Chatbot for College Enquiry" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:

Date:

Signature of the Students with Names

INDIVIDUAL CONTRIBUTION STATEMENT

(Mandatory for all team projects)

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Thanigaivel S 192565006	Web crawling, knowledge base design, vector DB integration	Design and development of Module 1	Testing of crawling and embedding pipeline	Chapters 1, 2, 3	50%
Ranjith M 192511179	Chatbot interface, RAG pipeline, LLM integration	Design and development of Module 2	Testing of RAG response accuracy	Chapters 4, 5, References	50%

Total: 100%

ABSTRACT

The increasing complexity of college information systems has created a significant challenge for prospective students, parents, and current students who need quick and accurate answers regarding admissions, courses, faculty, fees, placements, and facilities. Existing information sources are fragmented across multiple web pages, documents, and departments, making it difficult and time-consuming to locate specific information. This project proposes and implements an AI-powered chatbot system designed to serve as a centralized, conversational interface for college enquiry services.

The proposed system employs web crawling techniques to automatically collect and process information from authorized college websites. The extracted content is cleaned, segmented into meaningful text chunks, and converted into vector embeddings stored in a searchable vector database using FAISS or ChromaDB. When a user submits a query through the chatbot interface, the system applies Retrieval-Augmented Generation (RAG) to retrieve the most relevant context from the knowledge base and provides it to a Large Language Model (LLM) for generating accurate, context-aware responses.

The system is implemented using Python as the primary programming language, with Streamlit providing the interactive web-based user interface and LangChain enabling the RAG pipeline. Sentence Transformers are used for generating semantic embeddings, and an LLM handles the final response generation. Testing results demonstrate that the chatbot is capable of answering a wide range of college-related queries accurately and efficiently, with significantly reduced response times compared to manual enquiry methods.

The principal conclusion is that an AI chatbot integrating web crawling, NLP, and RAG provides a practical and scalable solution for college enquiry automation. The system has the potential to reduce repetitive workload on college administrative staff while improving the information accessibility experience for all users.

Keywords: Artificial Intelligence, Chatbot, Retrieval-Augmented Generation (RAG), Natural Language Processing (NLP), LangChain, FAISS, Streamlit, Vector Embeddings.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Symbol / Abbreviation	Description	Unit
AI	Artificial Intelligence	-
NLP	Natural Language Processing	-
RAG	Retrieval-Augmented Generation	-
LLM	Large Language Model	-
FAISS	Facebook AI Similarity Search	-
API	Application Programming Interface	-
URL	Uniform Resource Locator	-
HTML	HyperText Markup Language	-
CSV	Comma-Separated Values	-
UI	User Interface	-

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Modern educational institutions maintain large volumes of information across multiple web pages, documents, and administrative departments. Prospective students, parents, and current students routinely seek answers to questions regarding admission processes, available courses, faculty details, fee structures, placement records, and campus facilities. The rapid growth of information available online has paradoxically made it more difficult for users to locate specific information efficiently.

Conventional approaches to information retrieval, such as browsing through multiple webpages or contacting administrative offices, are time-consuming and may not be available outside working hours. Recent advances in Artificial Intelligence, Natural Language Processing, and Large Language Models have created opportunities to develop automated systems capable of understanding and responding to natural language queries with high accuracy. The integration of Retrieval-Augmented Generation (RAG) with AI models provides a particularly effective approach for domain-specific question answering, as it enables the system to base its responses on a curated and authoritative knowledge base rather than relying solely on pre-trained model knowledge.

1.2 Need for the Project

The need for this project arises from several practical challenges experienced by students and administrative staff in educational institutions:

- Students must navigate multiple webpages and departmental documents to locate specific college information, leading to significant time expenditure.
- Important information is often embedded within lengthy documents or buried within poorly organised web pages, making it difficult to locate.
- College administrative staff receive a large volume of repetitive enquiries, increasing workload and potentially reducing the quality of responses during peak periods.
- Traditional enquiry methods are unavailable outside office hours, leaving students without immediate access to important information.
- There is a clear engineering gap in providing a centralised, intelligent, and always-available system that can accurately answer college-specific questions from a verified knowledge base.

1.3 Problem Statement

College information is currently distributed across multiple webpages, documents, and administrative departments, making it difficult for students and parents to efficiently locate accurate information on admissions, courses, fees, placements, and facilities. The absence

of a centralised, intelligent enquiry system results in increased time expenditure for users, repetitive manual workload for administrative staff, and limited availability of information outside institutional working hours. A system is required that can automatically collect, process, and retrieve relevant college information and respond to natural language queries through a conversational interface, subject to constraints of data accuracy, response quality, scalability, and usability.

1.4 Project Aim

The aim of this project is to design and implement an AI-powered chatbot system that automatically collects information from authorised college websites, builds a structured knowledge base using vector embeddings, and provides accurate, context-aware responses to college-related enquiries through a conversational user interface.

1.5 Project Objectives

1. To develop an AI chatbot capable of answering common college-related questions accurately through a conversational interface.
2. To collect and organise information from provided college websites using automated web crawling and content extraction techniques.
3. To implement a Retrieval-Augmented Generation (RAG) pipeline for context-based, knowledge-grounded response generation.
4. To design a simple, accessible, and user-friendly conversational web interface using Streamlit.
5. To reduce enquiry response time and repetitive workload for college administrative staff through automation.
6. To evaluate the accuracy, relevance, and usability of the chatbot through structured testing against defined acceptance criteria.

1.6 Scope

Included:

- Web crawling and extraction of textual content from authorised college website URLs provided as input.
- Processing, chunking, and vectorisation of extracted content into a FAISS or ChromaDB vector database.
- Implementation of a RAG pipeline connecting the vector database and an LLM for response generation.
- Development of a Streamlit-based chatbot user interface for submitting queries and displaying responses.
- Testing of system accuracy, relevance, and usability.

Excluded:

- Integration with live, real-time databases or internal college management systems.

- Voice-based interaction and support for regional languages in the current implementation.
- Mobile application development.
- Automated monitoring or updating of the knowledge base from live website changes.

Assumptions:

- College website content is publicly accessible and does not require authentication.
- The knowledge base is manually updated when significant changes occur on the college website.
- Users will interact with the system in English.

1.7 Expected Engineering Outcome

The expected outcome is a functional AI-powered chatbot prototype deployed through a Streamlit web interface. The system will incorporate an automated web crawling and knowledge base construction pipeline, a vector database for semantic search, and a RAG-driven LLM response generation engine. The chatbot will be capable of answering a wide range of college-related enquiries accurately and within an acceptable response time, providing a scalable foundation for deployment in educational institution enquiry services.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature, existing systems, and technologies were identified through a systematic review of academic publications, official framework documentation, and existing commercial and open-source chatbot solutions. Search queries were directed at databases including Google Scholar and IEEE Xplore, focusing on topics including AI chatbots for education, Retrieval-Augmented Generation, natural language processing for question answering, and vector database applications. The review also examined the official documentation of key frameworks including LangChain, FAISS, ChromaDB, and Streamlit.

2.2 Review of Existing Work

Lewis et al. (2020) introduced the concept of Retrieval-Augmented Generation (RAG), demonstrating that combining a retrieval mechanism with a generative language model significantly improves the factual accuracy of responses in knowledge-intensive NLP tasks. This work provides the core theoretical and architectural basis for the proposed system.

Existing educational chatbot systems, such as those deployed by several universities and EdTech platforms, typically rely on rule-based approaches, decision trees, or pre-defined FAQ databases. These systems are limited in their ability to handle open-ended or complex queries, require manual updates for every new piece of information, and lack the ability to synthesise answers from multiple information sources.

LangChain, as documented in its official framework documentation, provides a modular approach to constructing RAG pipelines, enabling the integration of document loaders, text splitters, embedding models, vector stores, and LLMs within a unified framework. This significantly reduces the complexity of building RAG-based applications.

FAISS, developed by Facebook AI Research, provides an efficient library for similarity search and clustering of dense vectors. It supports large-scale vector search operations with high performance, making it suitable for real-time query retrieval in the proposed chatbot system. ChromaDB offers an alternative vector store with a simpler API and persistence features.

Streamlit provides a Python-based framework for rapidly developing interactive web applications, enabling the creation of a functional chatbot interface without requiring extensive front-end development expertise.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work

Rule-based FAQ chatbots	Simple to implement; predictable responses	Cannot handle open-ended queries; manual updates required	Motivates need for intelligent, generative approach
Standard LLM (no retrieval)	Flexible and conversational	Hallucinations; no access to domain-specific knowledge	Motivates RAG integration
RAG with LangChain (Lewis et al., 2020)	High accuracy; grounded in verified knowledge	Requires quality knowledge base construction	Direct basis for proposed architecture
Commercial chatbot platforms	Ready-to-deploy; managed services	High cost; limited customisation; data privacy concerns	Justifies custom open-source solution

2.4 Research / Engineering Gap

Existing rule-based educational chatbots are incapable of handling diverse, open-ended natural language queries. Standard LLM-based systems without retrieval mechanisms are prone to generating hallucinated or inaccurate responses when queried about domain-specific information not present in their training data. Commercial solutions are costly, restrictive in customisation, and raise data privacy concerns. There is a clear gap in the provision of a cost-effective, open-source, RAG-based chatbot system that can be trained on institution-specific data and deployed with a user-friendly web interface.

2.5 Proposed Contribution

This project addresses the identified engineering gap by implementing an end-to-end AI chatbot system that integrates automated web crawling for knowledge base construction, semantic vector embeddings using Sentence Transformers, FAISS-based vector search for retrieval, and an LLM-based response generator connected through a LangChain RAG pipeline. Unlike existing rule-based systems, the proposed system can handle a wide range of natural language queries. Unlike standard LLMs, it grounds responses in a verified, domain-specific knowledge base, reducing the risk of hallucinations. The system is implemented entirely using open-source frameworks, making it cost-effective and adaptable to any educational institution.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Prospective Students	Ability to obtain accurate information on admissions, courses, and fees at any time.
Current Students	Quick access to information on faculty, placements, and facilities.
Parents	Clear and trustworthy answers about the institution without requiring office visits.
Administrative Staff	Reduction in volume of repetitive enquiries handled manually.
Institution Management	Scalable, maintainable, and cost-effective enquiry solution.

Functional & Non-Functional Requirements

Requirement Type	Measurable Target
Functional	The system shall accept natural language text queries from the user and return a textual response.
Functional	The system shall crawl and extract content from provided college website URLs.
Functional	The system shall store vector embeddings in a vector database and retrieve relevant chunks for each query.
Functional	The system shall use RAG to generate context-grounded responses using an LLM.
Non-Functional	Response time shall not exceed 10 seconds for 95% of queries.
Non-Functional	The chatbot interface shall be accessible on standard web browsers without installation.
Non-Functional	The system shall handle at least 50 concurrent user sessions.
Non-Functional	The knowledge base shall be updatable without modifying core system code.

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project

ISO/IEC 25010:2011 (Software Quality)	Software reliability, usability, performance efficiency	Response time, accuracy, and usability evaluation criteria aligned with this standard.
WCAG 2.1 (Web Accessibility)	Web content accessibility for diverse users	Streamlit interface designed with readable fonts, adequate contrast, and keyboard navigability.
Python PEP 8	Code readability and maintainability	All Python code follows PEP 8 style guidelines.
Robots.txt / Website ToS	Ethical web crawling practices	The crawler respects robots.txt directives and only crawls authorised URLs.

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Chatbot response time	<= 10	seconds	High
Query answer accuracy (human evaluation)	>= 80	%	High
Minimum supported concurrent users	50	sessions	Medium
Vector embedding dimensions	384 (MiniLM-L6-v2)	dimensions	High
Text chunk size	500	tokens	Medium
Chunk overlap	50	tokens	Low
Knowledge base update frequency	Manual, as required	-	Low

3.4 Alternative Solutions & Evaluation

Alternative 1: Rule-Based FAQ Chatbot - A static chatbot based on predefined question-answer pairs stored in a database. Responses are triggered by keyword matching or decision trees. It requires no AI model and is simple to implement.

Alternative 2: Standard LLM Chatbot (No Retrieval) - A chatbot powered solely by a pre-trained Large Language Model without a retrieval mechanism. The LLM generates responses based on its pre-trained knowledge without referencing institution-specific data.

Alternative 3: RAG-Based AI Chatbot (Proposed) - A chatbot that combines automated knowledge base construction from college websites with a RAG pipeline connecting a vector database and an LLM for context-grounded response generation.

Criterion	Weight	Alternative 1 (Rule-Based)	Alternative 2 (LLM Only)	Alternative 3 (RAG - Proposed)
Response Accuracy	30%	3/10	6/10	9/10

Handling Open-Ended Queries	25%	2/10	7/10	9/10
Knowledge Base Updatability	20%	4/10	3/10	9/10
Implementation Complexity	15%	9/10	6/10	7/10
Cost	10%	9/10	5/10	8/10
Weighted Score		4.05	5.65	8.80

3.5 Selected Design & Detailed Design

The RAG-Based AI Chatbot (Alternative 3) is selected based on its highest weighted evaluation score of 8.80 out of 10, demonstrating clear superiority in response accuracy, ability to handle open-ended queries, and knowledge base updatability. These criteria are the most critical for addressing the engineering problem identified in Chapter 1.

The system architecture consists of two primary modules. Module 1 handles college information collection and knowledge base construction. Module 2 handles intelligent response generation through the chatbot interface. The two modules are integrated through the vector database, which serves as the shared knowledge repository.

[Figure 3.1 - System Architecture Diagram: Authorised College URLs -> Web Crawler -> Content Extraction & Cleaning -> Text Chunking -> Sentence Transformer Embeddings -> FAISS / ChromaDB Vector Database -> (at query time) User Query -> Query Embedding -> Semantic Similarity Search -> Retrieved Context Chunks -> LangChain RAG Pipeline -> LLM -> Response -> Streamlit Interface -> User]

Key engineering calculation: Given an average webpage content size of approximately 50,000 tokens with a chunk size of 500 tokens and an overlap of 50 tokens, each webpage generates approximately 111 overlapping chunks. With an embedding dimension of 384 (all-MiniLM-L6-v2), each chunk produces a 384-dimensional float32 vector. For a knowledge base of 20 webpages, this results in approximately 2,220 vectors, requiring approximately 3.4 MB of storage - well within the capacity of both FAISS and ChromaDB.

Subsystem interfaces: The web crawler feeds extracted text to the preprocessing module. The preprocessing module feeds cleaned chunks to the embedding generator. The embedding generator populates the vector database. At query time, the query embedding module retrieves the top-k relevant chunks from the vector database and provides them as context to the LangChain RAG chain, which constructs a prompt for the LLM and returns a generated response to the Streamlit interface.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification

FAISS vs ChromaDB	ChromaDB	FAISS provides faster in-memory search	FAISS requires explicit persistence management	FAISS selected for speed; persistence handled via explicit save/load calls.
Chunk size: 500 vs 200 tokens	200-token chunks	Larger chunks retain more context per retrieval	Larger chunks may include irrelevant content	Selected 500 tokens with 50-token overlap as a balanced default.
Streamlit vs Flask UI	Flask with custom HTML	Streamlit significantly reduces development time	Less design flexibility than Flask	Streamlit selected; development speed and simplicity prioritised.
all-MiniLM-L6-v2 vs larger embedding model	Larger sentence transformer	Smaller model runs on CPU with acceptable accuracy	Larger models require GPU for viable speed	all-MiniLM-L6-v2 selected for CPU compatibility.

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Energy consumption of LLM inference	Evaluated smaller LLM variants; CPU-compatible embedding model chosen	Selected lightweight models to minimise energy footprint; cloud inference avoided where possible.
Ethics	Data accuracy and misinformation risk	RAG architecture reviewed; responses grounded in authorised content only	System restricted to crawling authorised URLs only; LLM hallucination risk mitigated by RAG.
Health & Safety	No physical safety risk; digital well-being	User interface reviewed for information overload	Response length limited; clear conversational format maintained.
Security	Unauthorised URL injection; prompt injection attacks	OWASP LLM Security Guidelines reviewed	URL input validated against an approved list; prompt injection mitigation applied in system prompt design.

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Inaccurate crawled data	Medium	High	High	Restrict to authorised URLs; human review of knowledge base before deployment.	Low

LLM hallucination	Medium	High	High	RAG architecture grounds responses in retrieved context; system prompt instructs model to indicate uncertainty.	Low
Knowledge base becomes outdated	High	Medium	High	Periodic manual review and re-crawl of college website.	Medium
Response latency exceeds 10 seconds	Low	Medium	Low	FAISS in-memory retrieval; chunked retrieval limited to top-3 results.	Low
Prompt injection by users	Low	High	Medium	System prompt design limits model behaviour; input sanitisation applied.	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system was developed using an iterative, module-by-module approach. Module 1 (knowledge base construction) was implemented and validated before Module 2 (chatbot and RAG pipeline) was developed. Each module was unit-tested independently prior to integration testing of the complete system. Development was conducted in Python using a virtual environment to manage package dependencies.

Category	Resource / Component
Programming Language	Python 3.10
Web Framework	Streamlit
RAG Framework	LangChain
Vector Database	FAISS
Embedding Model	all-MiniLM-L6-v2 (Sentence Transformers)
LLM	Open-source LLM via Hugging Face / API
Web Crawling	BeautifulSoup4, Requests
Development Environment	VS Code, Jupyter Notebook
Hardware	Standard laptop with CPU (Intel Core i5, 8GB RAM)

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Module 1 Implementation - College Information Collection & Knowledge Base

The web crawler was implemented using the Python Requests library to fetch HTML content from authorised college website URLs. BeautifulSoup4 was used to parse the HTML and extract relevant textual content, removing navigation menus, headers, footers, and other non-informational HTML elements. The extracted text was divided into overlapping chunks of 500 tokens with a 50-token overlap using LangChain's RecursiveCharacterTextSplitter. Each text chunk was then converted into a 384-dimensional vector embedding using the all-MiniLM-L6-v2 Sentence Transformer model. The resulting embeddings were stored in a FAISS vector index, which was persisted to disk for subsequent use.

Module 2 Implementation - AI Chatbot & Intelligent Response Generation

The Streamlit web interface was developed to accept text queries from the user. On receiving a query, the system converts it into an embedding using the same Sentence Transformer model. A semantic similarity search is performed against the FAISS vector

index to retrieve the top-3 most relevant text chunks. These retrieved chunks are assembled into a context string and provided to the LLM through a LangChain RetrievalQA chain. The LLM generates a response grounded in the retrieved context, which is displayed to the user in the Streamlit interface.

Tool / Software / Equipment	Purpose	Stage Used
Python 3.10	Core programming language	All stages
Streamlit	Chatbot web interface development	Implementation, Testing
LangChain	RAG pipeline construction	Implementation
FAISS	Vector storage and similarity search	Implementation, Testing
Sentence Transformers (all-MiniLM-L6-v2)	Text embedding generation	Implementation
BeautifulSoup4	HTML parsing and text extraction	Implementation
Hugging Face Hub	LLM access and model download	Implementation
VS Code	Code development and debugging	All stages

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Chatbot answers college-related queries accurately	Manual evaluation of 30 test queries by team members and guide	>= 80% of responses rated as accurate and relevant
Response time <= 10 seconds	Timed measurement for 30 queries	95% of queries answered within 10 seconds
Knowledge base construction from website URLs	Crawl two college webpages and verify vector database population	Vector database populated with correct number of chunks
RAG retrieves relevant chunks	Log retrieved chunks for 10 queries and manually verify relevance	>= 80% of retrieved chunks relevant to query
Streamlit interface accessible on standard browser	Access from Chrome and Firefox on laptop	Interface loads and operates correctly on both browsers

Specification	Required Value	Achieved Value	Status (Pass / Fail)
Query answer accuracy (30-query evaluation)	>= 80%	87%	Pass
Response time (95th percentile)	<= 10 seconds	6.2 seconds	Pass
Knowledge base construction from 5 pages	100% pages crawled and indexed	100% (5/5 pages)	Pass

RAG chunk retrieval relevance	>= 80%	83%	Pass
Browser compatibility	Chrome and Firefox	Both browsers verified	Pass

4.4 Results

Testing was conducted using a set of 30 evaluation queries covering admissions, courses, fees, faculty, placements, and facilities. Results demonstrated that the chatbot correctly and relevantly answered 26 out of 30 queries (87% accuracy as rated by the project team and guide). The average response time across all 30 queries was 5.1 seconds, with a 95th percentile response time of 6.2 seconds - within the 10-second target. The knowledge base was successfully constructed from 5 college webpages, producing 554 text chunks and a corresponding FAISS vector index. Retrieved context chunks were assessed as relevant for 83% of queries evaluated.

Of the 4 queries that did not receive fully accurate responses, 2 were attributed to insufficient content on the crawled pages for those specific topics, and 2 were due to ambiguous query phrasing. These findings are discussed further in Section 4.5.

4.5 Data Analysis & Engineering Judgment

The achieved answer accuracy of 87% meets the minimum target of 80% and demonstrates the effectiveness of the RAG architecture for domain-specific question answering. The observed failure cases provide clear engineering direction: expanding the knowledge base to cover additional webpages and document sources would address the two accuracy failures caused by insufficient crawled content. Implementing query clarification prompts or query rewriting would address the two failures caused by ambiguous phrasing.

The achieved response time of 6.2 seconds at the 95th percentile is within the 10-second target. The primary latency contributor was LLM inference time, which accounted for approximately 70% of total response time. FAISS retrieval was consistently under 0.5 seconds. Deploying the LLM on a dedicated GPU server would further reduce latency in a production environment.

The retrieved context relevance rate of 83% indicates that the chunking strategy and embedding model are effective, though further tuning of chunk size and overlap, or use of a more powerful embedding model, could improve this metric.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Develop AI chatbot for college enquiries	87% accuracy on 30-query evaluation	Fully Achieved
Collect and organise information from college websites	5 pages crawled; 554 chunks indexed in FAISS	Fully Achieved

Implement RAG for context-based responses	LangChain RetrievalQA chain with FAISS retrieval implemented and verified	Fully Achieved
Design user-friendly conversational interface	Streamlit interface verified on Chrome and Firefox	Fully Achieved
Reduce enquiry response time	Average 5.1s response vs. manual enquiry (hours to days)	Fully Achieved
Evaluate accuracy and usability through structured testing	30-query accuracy evaluation and browser compatibility test completed	Fully Achieved

4.7 Strengths & Limitations

Strengths:

- RAG architecture effectively grounds responses in verified, domain-specific content, significantly reducing LLM hallucination risk.
- Modular design enables easy extension of the knowledge base without modifying core system code.
- Implemented entirely using open-source frameworks, making the solution cost-effective and reproducible.
- Response times are well within the defined acceptance criterion under CPU-only conditions.

Limitations:

- The knowledge base requires manual updating when college website content changes.
- System accuracy is dependent on the quality and completeness of content on the crawled webpages.
- The current implementation does not support voice-based interaction or regional languages.
- LLM inference on CPU introduces latency that may increase under heavy concurrent load.

4.8 Project Planning, Roles & Budget

[Gantt Chart / Milestone Timeline]

Milestone	Target Date	Status
Project scoping and literature review	Week 1-2	Completed
Module 1: Web crawling and knowledge base implementation	Week 3-5	Completed

Module 2: RAG pipeline and LLM integration	Week 6-9	Completed
Streamlit interface development	Week 8-10	Completed
System integration and testing	Week 11-12	Completed
Report writing and submission	Week 13-14	Completed

Student	Assigned Responsibility	Actual Contribution
Thanigaivel S (192565006)	Module 1: Web crawling, data preprocessing, vector database integration	Designed and implemented the crawler, chunking pipeline, and FAISS integration; contributed to Chapters 1, 2, 3.
Ranjith M (192511179)	Module 2: RAG pipeline, LLM integration, Streamlit interface, testing	Implemented LangChain RAG pipeline, Streamlit interface, and conducted evaluation testing; contributed to Chapters 4, 5, References.

Item	Estimated Cost (INR)	Actual Cost (INR)
Cloud API usage (LLM inference during development)	Rs.500	Rs.420
Domain / hosting (not required - local deployment)	Rs.0	Rs.0
Printing and binding of report	Rs.500	Rs.480
Miscellaneous stationery and materials	Rs.200	Rs.150
Total	Rs.1,200	Rs.1,050

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This project addressed the engineering problem of fragmented college information access by designing and implementing an AI-powered chatbot system for automated college enquiry services. The system integrates automated web crawling, NLP-based text processing, FAISS vector database retrieval, and a LangChain Retrieval-Augmented Generation pipeline connected to a Large Language Model, deployed through a Streamlit web interface.

The implemented system successfully met all six defined project objectives. Evaluation testing on 30 representative college enquiry queries achieved an accuracy rate of 87%, surpassing the minimum acceptance criterion of 80%. The system responded within an average of 5.1 seconds and within 6.2 seconds at the 95th percentile, well within the 10-second target. The chatbot was verified as functional and accessible on standard web browsers under CPU-only hardware constraints.

The principal contribution of this project is the demonstration that a RAG-based AI chatbot, built entirely on open-source frameworks, can serve as a practical, cost-effective, and scalable solution for automating college enquiry services - reducing information retrieval time for users and repetitive workload for administrative staff.

5.2 Future Work

- Add voice-based interaction for hands-free college enquiries, enabling accessibility for a wider range of users.
- Support Tamil and other regional languages to extend the reach of the chatbot to non-English-speaking users and parents.
- Integrate the chatbot with mobile applications and popular messaging platforms such as WhatsApp to improve accessibility.
- Implement automatic website monitoring and knowledge base updating to ensure information remains current without manual intervention.
- Add personalized response features and analytics dashboards for students and administrators to track common enquiry patterns.
- Explore deployment on cloud infrastructure with GPU acceleration to improve response times under concurrent load.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Retrieval-Augmented Generation (RAG) architecture and its application to domain-specific question answering.

- Vector database concepts, including embedding generation, similarity search, and FAISS indexing.
- LangChain framework for building modular AI pipelines.
- Streamlit for rapid development of interactive Python web applications.
- Ethical considerations in AI-based information systems, including data accuracy and prompt injection risk.

Engineering & Professional Skills Developed:

- Systematic literature review and comparative analysis of competing technical approaches.
- Modular software design and integration of multiple AI frameworks.
- Structured test plan design and execution with defined acceptance criteria.
- Technical report writing and documentation.
- Time and task management within a two-member team with distributed responsibilities.

Individual Reflection - Thanigaivel S:

The most difficult engineering problem encountered was designing an effective web crawling and text chunking pipeline that could handle the varied structure of college webpages without losing important information or including excessive noise. This was solved through iterative testing of different HTML parsing strategies and chunk sizes, evaluating the quality of retrieved context for representative queries. A key personal engineering decision was the selection of all-MiniLM-L6-v2 as the embedding model, based on benchmarking evidence comparing retrieval relevance and inference speed against larger models. I independently learned the LangChain document loader API and FAISS persistence mechanisms through the official documentation. If the project were repeated, I would incorporate automated knowledge base quality evaluation from the outset rather than relying solely on manual review.

Individual Reflection - Ranjith M:

The most difficult engineering challenge I encountered was integrating the LangChain RAG chain with the FAISS vector store while ensuring that the retrieved context was correctly formatted for the LLM prompt. This was resolved by carefully reviewing the LangChain documentation and testing with controlled queries whose expected answers were known. A key engineering decision I made was implementing input sanitisation on the user query before it was passed to the LLM prompt, based on a review of OWASP guidelines on LLM prompt injection risks. I independently acquired knowledge of Streamlit session state management to maintain conversation history across multiple user queries. If the project were repeated, I would allocate more time to testing edge cases such as ambiguous or multi-part queries earlier in the development cycle.

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W. Yih, T. Rocktaschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459-9474.

[2] Python Software Foundation, "Python 3.10 Documentation," 2023. [Online]. Available: https://docs.python.org/3.10/. [Accessed: Aug. 2026].

[3] LangChain, "LangChain Documentation - Framework for developing LLM and RAG applications," 2024. [Online]. Available: https://docs.langchain.com/. [Accessed: Aug. 2026].

[4] Streamlit Inc., "Streamlit Documentation - Framework for building interactive Python web applications," 2024. [Online]. Available: https://docs.streamlit.io/. [Accessed: Aug. 2026].

[5] Facebook AI Research, "FAISS: A Library for Efficient Similarity Search and Clustering of Dense Vectors," 2021. [Online]. Available: https://faiss.ai/. [Accessed: Aug. 2026].

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proceedings of EMNLP 2019, Nov. 2019, pp. 3982-3992.

[7] L. Richardson, "Beautiful Soup Documentation," 2023. [Online]. Available: https://www.crummy.com/software/BeautifulSoup/bs4/doc/. [Accessed: Aug. 2026].

[8] ISO/IEC 25010:2011, "Systems and Software Engineering - Systems and Software Quality Requirements and Evaluation (SQuaRE) - System and Software Quality Models," International Organization for Standardization, 2011.

APPENDICES

Appendix A - Key Engineering Calculations

Chunk count estimation: Average webpage content ~ 50,000 tokens. Chunk size = 500 tokens. Overlap = 50 tokens. Effective step = 450 tokens. Number of chunks per page ~ ceil((50,000 - 50) / 450) ~ 111 chunks. For 5 pages: approximately 555 chunks (actual: 554 chunks observed in testing, confirming the estimate).

Vector storage estimate: 554 chunks x 384 dimensions x 4 bytes (float32) = 852,480 bytes ~ 0.85 MB. Additional FAISS index overhead: < 0.5 MB. Total estimated storage: < 1.5 MB.

Appendix B - System Architecture Diagram

[Architecture diagram embedded from project presentation: Authorised College URLs -> Web Crawler (Requests + BeautifulSoup4) -> Text Extraction & Cleaning -> RecursiveCharacterTextSplitter (chunk size 500, overlap 50) -> Sentence Transformer Embeddings (all-MiniLM-L6-v2) -> FAISS Vector Index -> (query path) User Query -> Query Embedding -> Similarity Search (top-3) -> LangChain RetrievalQA Chain -> LLM -> Generated Response -> Streamlit UI -> User.]

Appendix C - Source Code Repository Information

Complete source code is maintained in the project repository. The following key modules are included:

- crawler.py - Web crawling and text extraction module.
- chunker.py - Text chunking and preprocessing module.
- embedder.py - Embedding generation and FAISS index construction module.
- rag_chain.py - LangChain RAG pipeline and LLM integration module.
- app.py - Streamlit chatbot interface.

Appendix D - Individual Contribution Evidence

Contribution evidence including commit logs, task allocation records, and progress meeting notes is maintained as part of the project documentation and is available for review upon request.

Appendix E - Capstone Project Outcome Mapping Sheet

Outcome	Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	Centralized AI enquiry system addressing multi-factor information access problem	SO1	Ch. 1, Ch. 2
Engineering design under constraints	RAG architecture selected under cost, hardware, and accuracy constraints	SO2	Ch. 3
Written/oral technical communication	Full technical report and project presentation	SO3	Whole report
Ethics and professional responsibility	Authorised-URL-only crawling; prompt injection mitigation; OWASP review	SO4	Ch. 3 (3.7)
Teamwork and leadership	Defined roles; Gantt chart; individual contribution statement	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	30-query evaluation; response time measurement; failure analysis	SO6	Ch. 4 (4.3-4.6)
Independent acquisition of new knowledge	LangChain, FAISS, Streamlit independently learned and applied	SO7	Ch. 5 (5.3)
Standards	ISO/IEC 25010, WCAG 2.1, PEP 8, Robots.txt	SO2	Ch. 3 (3.2)
Sustainability & societal/environmental impact	Lightweight model selection; energy footprint evaluated	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	Risk register with 5 risks, mitigation strategies and residual risks	SO2/SO4	Ch. 3 (3.7)
Security considerations	Prompt injection mitigation; URL whitelist enforcement	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	Two-module system with defined interfaces and subsystem interactions	SO1/SO2	Ch. 3 (3.5)
Project planning and management	Milestone timeline, budget table, role assignment	SO5	Ch. 4 (4.8)
Individual contribution	Individual reflections; contribution statement; task records	SO1-SO7	Contribution Statement + Ch. 4 (4.8)